Documento técnico del proyecto integrador

GitHub Actions, Docker y Microsoft SQL Server

Proyecto: Automatización y pruebas de `EmpresaDB`
Repositorio: `proyecto-sqlserver-actions`
Autor: [Escribe aquí tu nombre]
Fecha: [Fecha de entrega]

1. Introducción y objetivo

El objetivo es automatizar el inicio de un servidor SQL Server, el despliegue de una base de datos, la creación de objetos de negocio y su validación. Los archivos residen en un repositorio GitHub; GitHub Actions ejecuta un contenedor Docker temporal con SQL Server 2022 Developer y aplica los scripts T-SQL. Un segundo workflow, independiente y programado, genera un reporte CSV de ventas como Artifact.

2. Tecnologías involucradas

Git y GitHub. Git permite controlar versiones de los archivos; GitHub aloja el repositorio, su README, los workflows y sus resultados.

GitHub Actions. Automatiza procesos definidos en archivos YAML dentro de `.github/workflows`. `deploy.yml` se ejecuta al hacer `push` a `main` o manualmente. `reporte.yml` se ejecuta con `workflow_dispatch` o cron los lunes a las 14:17 UTC. Cada ejecución usa una máquina Ubuntu temporal de GitHub.

Docker. Proporciona un contenedor aislado basado en la imagen oficial `mcr.microsoft.com/mssql/server:2022-latest`. No se instala SQL Server en la computadora del estudiante: el contenedor se ejecuta en el runner de GitHub.

Microsoft SQL Server y T-SQL. SQL Server es un gestor relacional; Transact-SQL permite crear bases, tablas, restricciones, procedimientos almacenados y triggers. `sqlcmd` envía los scripts desde el workflow a la instancia SQL.

GitHub Secrets. Almacena la contraseña del usuario `sa` como `MSSQL_SA_PASSWORD`, sin incluirla en archivos versionados. La variable antigua `SA_PASSWORD` está desaconsejada por Microsoft para el contenedor. `ACCEPT_EULA=Y` confirma la aceptación de la licencia y `MSSQL_PID=Developer` elige la edición de desarrollo.

Artifacts y CSV. `upload-artifact@v4` adjunta el reporte producido en una ejecución de Actions. CSV es un formato tabular de texto compatible con hojas de cálculo.

3. Diseño de la base de datos

EmpresaDB almacena tres tablas:

Tabla	Campos importantes	Propósito
`Ventas`	`VentaID`, `Producto`, `Cantidad`, `Fecha`	Registrar cada venta.
`AuditoriaVentas`	`AuditoriaID`, `VentaID`, `FechaRegistro`, `Mensaje`	Registrar automáticamente cada venta; tiene relación con `Ventas`.
`Inventario`	`Producto`, `Existencias`	Guardar unidades disponibles por producto.

`VentaID` y `AuditoriaID` son llaves primarias con `IDENTITY(1,1)`. Las fechas reciben `GETDATE()` por defecto. Se exige una cantidad positiva y un inventario no negativo mediante restricciones `CHECK`.

4. Procedimiento almacenado

`sp_RegistrarAuditoria` recibe un parámetro **de tipo tabla** (`dbo.TipoVentasInsertadas READONLY`) que contiene `VentaID`, `Producto` y `Cantidad` para todas las ventas de una misma instrucción. Primero agrupa las cantidades por producto. Después descuenta las unidades mediante un único `UPDATE` por conjunto sobre `Inventario`; si falta el producto o no hay

existencias suficientes, lanza un error, lo que provoca el rechazo de la inserción que activó el trigger. Por último inserta en `AuditoriaVentas` un registro por cada venta con el mensaje indicado en las instrucciones.

Se cambió la firma de `@VentaID INT` del ejemplo inicial porque solo permite identificar una venta; un parámetro de tipo tabla puede transportar todas las filas del conjunto y permite implementar ambos retos sin un bucle.

5. Trigger y tabla lógica `inserted`

`trg_AuditarVenta` se define `AFTER INSERT ON dbo.Ventas`. En SQL Server el trigger se ejecuta **una vez por instrucción**, no una vez por fila. La tabla lógica `inserted` contiene las filas nuevas de la instrucción de inserción; por ejemplo, si `INSERT ... VALUES (...) , (...) , (...)` inserta tres ventas, `inserted` contiene esas tres ventas durante la ejecución del trigger.

El trigger copia **todo** el conjunto de `inserted` a `@Ventas` y ejecuta `sp_RegistrarAuditoria @Ventas = @Ventas`. La tabla `inserted` no es una tabla física persistente y no debe suponerse que contenga una única fila.

6. Procesamiento fila por fila vs. por conjuntos

Fila por fila: un cursor o bucle procesa cada venta individualmente; con N ventas realiza N iteraciones, lo que incrementa la complejidad y los viajes lógicos entre operaciones. En el ejemplo inicial, `SELECT @VentaID = VentaID FROM inserted` solo guarda un ID y pierde las demás ventas cuando hay varias.

Por conjuntos (set-based): `INSERT ... SELECT` registra de una sola vez todas las auditorías; `GROUP BY Producto` suma las cantidades para descontar existencias; `UPDATE ... JOIN` modifica varios productos en una sola sentencia. Esto es más sencillo de comprobar, suele rendir mejor para conjuntos y evita el error de ignorar ventas adicionales.

7. Flujo de integración continua y seguridad

1. Un cambio a `main` o una ejecución manual inicia `deploy.yml`.
2. `actions/checkout@v4` obtiene los archivos.
3. `scripts/setup_db.sh` inicia Docker con la imagen oficial y el Secret inyectado como variable de entorno.
4. Un ciclo de espera verifica que `sqlcmd` pueda conectarse a SQL Server.
5. Se ejecutan, en orden, `create_database.sql`, `create_tables.sql`, `create_procedure.sql`, `create_trigger.sql` y `test_data.sql`.
6. `verify.sql` verifica las cinco ventas, cinco auditorías e inventario esperado; si algo falla, `THROW` hace fallar la ejecución.
7. `reporte.yml` crea una nueva instancia temporal, reconstruye los datos, exporta `reporte_ventas.csv` y lo publica como Artifact.

Nota de alcance: esto es un despliegue reproducible en un entorno efímero de integración continua, no un servidor SQL Server persistente accesible después de terminar el workflow. No debe confundirse con producción.

8. Datos de prueba y resultados esperados

Se inicializa el inventario con Laptop=10, Monitor=12 y Mouse=20. Primero se insertan las dos ventas exigidas: Laptop=3 y Monitor=5. Luego se hace **una sola instrucción multirregistro** con Laptop=2, Monitor=1 y Mouse=4.

Métrica	Resultado esperado
Total de ventas	5
Registros de auditoría	5
Laptop: 10 - 3 - 2	5
Monitor: 12 - 5 - 1	6
Mouse: 20 - 4	16
Líneas del reporte CSV	6 (cabecera + 5 ventas)

Los resultados son expectativas definidas en las verificaciones SQL; se deben confirmar con capturas reales de las ejecuciones de GitHub Actions.

9. Segundo workflow: CSV programado

El archivo `.github/workflows/reporte.yml` incluye `schedule` con `cron: '17 14 * * 1'`, correspondiente a lunes 14:17 UTC, y `workflow_dispatch` para lanzarlo a demanda. Como cada runner es temporal, el workflow vuelve a crear la base y datos. Exporta encabezado y filas con campos entrecomillados, y publica el archivo mediante `actions/upload-artifact@v4` con el nombre `reporte-ventas-csv` y retención solicitada de 30 días. Los trabajos programados pueden iniciarse con demora y solo operan en la rama predeterminada.

10. Comparación con el repositorio de referencia

La referencia `jesusrevilla/T41A-P16` usa PostgreSQL, un workflow de Actions, README y dependencias Python para sus pruebas. Este trabajo conserva el patrón de repositorio documentado y automatización de pruebas, pero utiliza las tecnologías obligatorias: **SQL Server + Docker + GitHub Secrets + T-SQL + dos workflows + Artifact CSV**. El ejemplo solo orienta la organización, no el motor ni la lógica de negocio.

11. Conclusión

La solución integra control de versiones, automatización reproducible, contenedores y validaciones de base de datos. El uso de `inserted` y parámetros de tipo tabla permite tratar ventas múltiples de forma correcta; el inventario se descuenta en el procedimiento almacenado y se mantiene una auditoría por venta. El reporte independiente demuestra la ejecución programada y publicación de resultados en GitHub Actions.

12. Fuentes

- Microsoft Learn: SQL Server en Docker. https://learn.microsoft.com/en-us/sql/linux/quickstart-install-connect-docker
- Microsoft Learn: Parámetros de tipo tabla. https://learn.microsoft.com/en-us/sql/relational-databases/tables/use-table-valued-parameters-database-engine
- Microsoft Learn: Utilidad sqlcmd. https://learn.microsoft.com/en-us/sql/tools/sqlcmd/sqlcmd-utility
- GitHub Docs: Eventos de GitHub Actions. https://docs.github.com/en/actions/reference/workflows-and-actions/events-that-trigger-workflows
- Repositorio de ejemplo del docente: https://github.com/jesusrevilla/T41A-P16